Building Online Judge From Scratch

8 min read · Dec 13, 2025

Bhimsen Kulkarni Follow

Listen Share

Continuing my series on implementing system-design problems from scratch, I will now build an online coding judge similar to LeetCode. This will be a three-part series: the first part presents the high-level design, the second covers the backend implementation, and the final part implements the UI.

In this blog, I will try to implement more than the core code-evaluation service. I will also build a discussion forum for solution conversations, an admin portal, competition management, and subscription handling. This broader scope will let me try different technologies and evaluate their tradeoffs.

As usual, I will build the application in Python. So without further ado, let's dive into the requirements and high-level design of our online judge.

Requirements

Requirements are a combination of functional requirements(what the system does) and non functional requirements(how the system behaves).

Functional Requirements

Functional requirements are derived from the user's journey. We have two actors here i.e. User and Admin. Since we have three components, we will divide the functional requirements amongst them.

Core Services

- Users should be able to list, sort, search, filter(based on difficulty, tags) and view problems.

- Users should be able to submit the code to the judge using any supported languages and get the verdict back(pass/fail).
- Users should be able to add custom test cases that should be used during test case run.
- Users should be able to view the **last 10 submissions** for a given problem. This is important. It means we only keep last 10 submissions per problem per user. This requirement will come handy when we do back of the envelope calculations.
- Users should be able to view why a submission failed and can see and download the test case for the failure scenario.
- Users should be able to subscribe(recurring, monthly, yearly, lifetime) to premium account to view premium services.
- Users should be able to view and participate(sign-up) in a contest.
- Users should be able to see the live leaderboard.

Admin Portal

Admin portal is a management portal which will be used to manage the resources such as problems, competitions, users etc.

- It should provide an interface to perform basic CRUD operations on all the resources.
- An admin can designate a user as problem setter or invite users as problem setter. Only problem setter and admin can create problems.
- Admin portal is only accessable to users with roles: admin & problem setter.
- An admin should be able to mark a problem premium or free.

Community Service

- Users should be able to add a post against a problem. This is done to explain their solution or for editorial purposes.
- Users should be able to comment on their posts. Nested comments are allowed (only one level nesting).
- Users should be able to upvote a post or comment to improve their ranking.

- Users should be able to update or delete their post or comment. If a post is deleted, the comments should still be there but only visible to the author from their profile until they explicitly delete them.

Non Functional Requirements

Non functional requirements talk about how a system behaves or performs. This involves answering questions like scalability(request rate, storage), CAP theorem(consistency, availability, partition tolerance), security, latency, concurrency, idempotency or uniqueness guarantees(for some problems) and in the end we do some back of the envelope calculations to get an idea or sense of the scale of our problem.

- Since we are running arbitrary code on our servers. Security is very important. So each user should run inside strict sandboxes and isolated containers. The code can't access the outside world nor the outside world contact the code.
- Users should be able to get their verdict back within maximum 5 seconds (normal user). A premium user should be able to see the results within max 3 seconds.
- All pages should load within under a second.
- During the contest, the system should be able to handle a high submission rate in a span of last 10 minutes to an hour without degrading the performance for normal users. So the system should support high write throughput.
- During the contest, the system should be scaled enough to process all the submitted jobs fast enough to return the response back within 2 or 3 seconds.

Traffic Estimation

Let's do a quick traffic estimation of reads and writes. This should allow us to choose the right database and other stateful solutions in our design. We will do the estimation for 100k DAU.

We make the following approximate assumptions:

For code submissions:

- For contests assume 4 submissions/user(worst case) in a short span like 5 minutes. And since each contest has around 4 to 5 problems each and each user

making 5 submissions (best case) and worst case 3x i.e. 15 submissions each within a 90 minute window of the contest.

- Under normal non contest scenarios, we expect 5 submissions in a day and worst case 10 submissions per day per user.
- Each submission including code and metadata, we expect it to be around 10kb max. With 8kb for code text.

For discussion service:

- We expect each user making 1 or 2 posts a day per user.
- We expect similar trends for comments to these posts.
- Each post can be as big as 40kb. And each comment is under 0.5kb.

For reads, I assume users read more often they write posts or submit their code. So overall we assume a 10:1 read/write ratio. So for every write, we expect 10 reads per user.

Get Bhimsen Kulkarni's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

100k Daily Active Users:

- Total submissions per day: 100k * 10 = 1M sub / day = 12 sub/sec
- During context: For 90 minutes: 100k * 15 = 1.5M in 90 minutes.= 277 sub/sec
- Last five minutes or burst: 100k * 4 = 400k in 5 minutes = 1.4k sub/sec.
- For community: 100k * 4 = 400k per day = 5 writes/sec.
- So total reads per day: (12 + 5) * 10 = 170 reads/second.

Storage Requirements:

Since problems and test cases are constant we will not worry about those. So we will mostly concentrate on submissions and community posts/comments storage requirements.

100k Daily Active Users:

- Submissions: 1M * 10kb = 10Gb/day and during contests = 20Gb/day. Since we don't run contests every day, we will assume 10Gb/day as storage requirement for submissions.
- For posts: 200k * 40kb = 8Gb/day.
- For comments: 200k * 0.5kb = 0.09Gb/day.

So per day we would need a maximum of ~20Gb of storage space. But in our functional requirement I stated that, the service will only keep the last 10 submissions for a given user and problem. So submission storage space would remain constant throughout the year.

Now the only thing that varies are the posts/comments from community service. We need around ~10Gb/day based on our traffic estimation and so the service needs around ~4Tb for a year.

Choosing Database

When choosing a database, we should evaluate a few scenarios before selecting one for our services. I use the following questions to guide the decision:

- Are our data models relational? Do they need normalization? Normalized tables help avoid duplicates and maintain strong data integrity.
- Do we need transaction support, relational integrity, or constraint enforcement that we do not want to implement in the application?
- Can we accept denormalized data? Denormalization improves read performance but makes writes more complex.
- Do we need to support ad hoc queries?
- Is the application read-heavy or write-heavy?

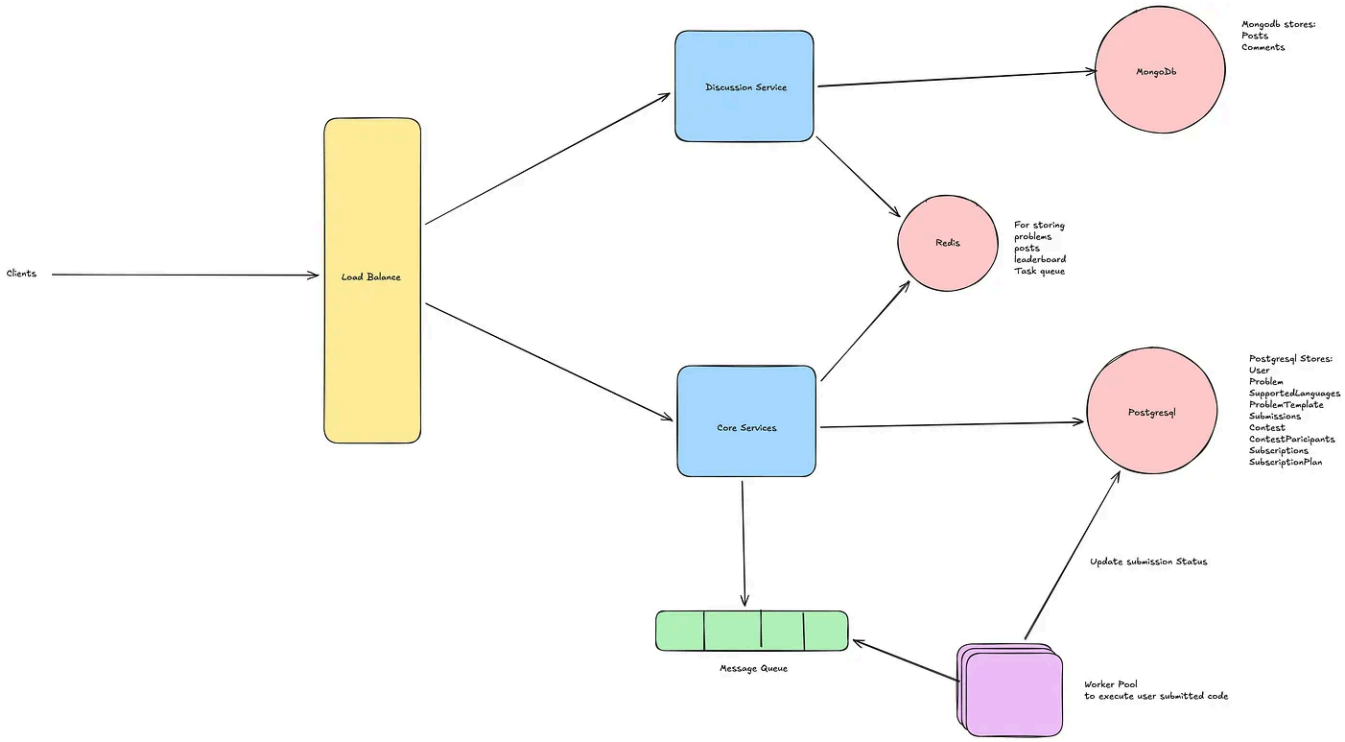
- What is the operational complexity? If we need sharding, do we want to manage it ourselves or rely on a service that handles it transparently?
- What are the cost implications?
- How easy is development? If multiple databases meet our requirements, prefer the one the team knows best.

Given our requirements and traffic estimates, most databases can handle our read and write throughput. Our core services require about 40 GB of storage at most. The discussion service will need much more storage and therefore should support sharding. Because the discussion schema may evolve, we should choose a schemaless database that supports sharding out of the box and exposes an API that hides sharding details so queries remain the same whether we shard or not.

Therefore, we will use **PostgreSQL** for our core services and **MongoDB** for the discussion service.

High Level Design

In high level design we need to come up with a design that satisfies all the functional requirements that we defined earlier and later run it through the non functional requirements and see if the system handles those constraints that we have mentioned.



Components

- **CoreServices**: This service implements bulk of our leetcode features. So when users try to see anything related to problems, want to submit solutions and fetch the status or history, it contacts the core service for it. It also exposes an admin portal that helps admins to create contests and add problems to the database. The core-services also take care of authentication/authorization of requests.

- **Discussion Service**: This service handles discussion forums per problem. So all the editorial, solution posts and their discussion is handled by the discussion service.

- **Worker containers**: These are worker containers that execute user submitted code. They read the submission requests from redis task queues and process them and update their status once completed. To get the metadata and update submission results, it will contact the core-services.

- **Redis**: Redis is used to provide task queue features and caching static pages such as problems page and discussion forum posts to reduce load on the databases. Redis is a good choice because it can handle the write and read throughput

Open in app ↗ Sign up Sign in

Medium Search

- **Postgresql**: It is used to store all our core entities like users, problems, test cases, submissions, contests etc.

- **Mongodb**: It is used to store all the forum related entities such as posts, comments. It is schemaless, which allows us to quickly implement features like tags, featured comments(recent, most-commented etc), It also allows us to shard transparently so if the discussion forum scales in the future due to data size, we don't have to worry about any data migration etc as it takes care of it transparently.

Executing Code Securely

I will use the shell utility **isolate** to execute code securely. **Isolate** is a standard sandbox tool for running user-submitted code and is lighter and faster than running code in Docker containers.

We will build a image that includes the **isolate** command and all supported language binaries. Worker machines boots up using that image. Workers will fetch the submitted source and its test cases, create the workspace files (`input.txt` ,

`output.txt`, `metadata.txt`), append the driver code from the problem template table to the source, and compile the code when the language requires it. The workers will then run the compiled or interpreted code inside **isolate**, compare the produced `output.txt` with the expected outputs from the test cases, and update the submission status accordingly.

Sample **isolate** command:

```
isolate \
  --box-id=<submission-id> \
  --run \
  --cg \ # use cgroups v2
  --cg-mem=262144 \ # total CGROUP memory limit for the box in KB (256 MB)
  -t 1.0 \ # CPU time limit in seconds (accounting CPU time)
  -w 3.0 \ # wall-clock timeout (a bit larger than -t)
  -x 0.1 \ # extra time to let process exit cleanly
  -p=1 \ # max child processes/threads allowed (1 = no forks)
  -f 10240 \ # per-file size limit in KB (10 MB)
  -k 8192 \ # stack limit in KB (8 MB)
  -o stdout.txt -i input.txt -r stderr.txt --meta meta.txt \
  -- /usr/bin/python3 main.py
```

Ok, now the high-level design is ready. Next up is the implementation. The service will be built with **FastAPI** for the HTTP API and **Celery** to interface with message queues and drive background workers. See you in the next blog post.

System Design Interview | System Design Concepts | Microservices | Design | Software Development

Follow

Written by Bhimsen Kulkarni

5 followers · 10 following